

IoT Challenge 2026 – Smart Healthcare with Edge AI

SMART GLASSES FOR ASSISTING BLIND USERS

INTEGRATED WITH EDGE AI AND COMPUTER VISION



Team name	Vua
Members	Ha Duc Khoi Nguyen Huong Giang Nguyen Minh Cat Kieu Nhat Linh
School/Class	VNU University of Engineering and Technology
Mentor	
Submission date	May 13, 2026

Contents

1	Background and problem statement	2
2	Project objectives	2
3	Proposed solution and usage flow	3
4	System architecture	5
4.1	Speech processing with edge AI on EFR32xG26	6
4.2	Communication protocol and HTTP Vision Server	8
4.2.1	BLE GATT between EFR32xG26 and the Android gateway	9
4.2.2	HTTP Vision Server on ESP32-CAM	10
4.2.3	Unified request from the Android gateway to the AI server	11
4.2.4	Time synchronization and session management	11
4.3	Hardware components	11
4.4	Software and AI flow	12
5	Resources, assignment, and schedule	13
6	Current results and progress of the prototype implementation	15
7	Conclusion	17

1. Background and problem statement

For blind users, daily difficulties are not limited to walking or avoiding obstacles. In many situations, what matters more is quickly understanding the context of the surrounding environment: what is in front, whether there is something noteworthy on the left or right, which path is clear, and whether there is a door, table, chair, or person standing ahead. Without such information, users often have to stop and probe the environment, move more slowly, and find it harder to make timely decisions in a changing space.

Existing assistive tools only address part of that need. A white cane is very useful for detecting low obstacles, but it cannot fully represent objects at head level, on tables, on shelves, near entrances, or on either side of the user. Global Positioning System (GPS) is suitable for large-scale navigation, but it is not detailed enough to answer practical questions such as which direction to go, whether there is an obstacle on the left, or where a needed object is located. Therefore, there is a need for a system that can observe the nearby environment, understand spatial context, and respond through audio in a short, easy-to-understand, and immediately usable way.

From that need, this project aims to build a system that assists blind users by combining speech and computer vision in a single interaction session. The system is designed not only to support movement, obstacle awareness, and environmental description, but also to enable vision-based question answering, helping users find and pick up objects more conveniently in daily life. For example, a user can ask what is in front, which side is free, where a water bottle is, or which direction to reach in order to take a needed object. This approach makes interaction with the environment more proactive, reduces dependence on a human helper for small daily tasks, and creates a technical problem suitable for a research and demonstration prototype.

2. Project objectives

The overall objective of this project is to build a smart glasses prototype that can support blind users in near-range and common daily needs, rather than focusing on only one isolated function. Specifically, the system aims at four main support groups: understanding surrounding context, vision-based question answering, finding and assisting with object pickup, and suggesting safe movement directions within a nearby range. To realize these goals, the prototype is implemented on four finalized components: EFR32xG26 for edge speech intelligence, ESP32-CAM for vision, an Android gateway for session coordination, and an AI server for heavy processing.

The technical objective of the project is to complete a closed processing chain in which the user can issue a voice command, the system captures an image at the right moment, synchronizes data by session, and generates a short and clear audio response. The implementation focuses on four main blocks: wake-word detection and end-of-speech detection on EFR32xG26; image provision through an HTTP vision server on ESP32-CAM; management of `session_id` and `timestamp` on the Android gateway; and speech recognition, object detection, and image analysis on the AI server. In this architecture, the vision-language model (VLM) plays an important role in combining the user question with the captured image to infer the appropriate context, answer questions about the scene, describe left-right positions, support object finding, and suggest short-term navigation directions. After the result is generated, it is sent back through BLE so that EFR32xG26 can play it as audio for the user. The final goal of the prototype is to create a seamless interaction experience from the moment the user says “Hey Tèo” until the user receives an audio response aligned with the real-world context.

To make the implementation intention clearer, the objectives below are grouped by the main technical blocks of the system rather than by general function only.

Table 1: Technical objectives by implementation block.

Technical block	Implementation focus	Expected outcome
System architecture	Complete a multi-node architecture consisting of EFR32xG26, ESP32-CAM, Android gateway, and AI server in one unified processing flow	A synchronized prototype from command reception to response playback
Edge speech AI	Detect wake word, record the command, and determine end-of-speech on EFR32xG26	A stable speech node that opens sessions at the right time and sends compact, context-appropriate data
Communication and synchronization	Maintain two-way BLE connection between EFR32xG26 and Android gateway, call HTTP from Android gateway to ESP32-CAM to fetch images, and let only the Android gateway send unified requests to the AI server while managing <code>session_id</code> and <code>timestamp</code> for each session	Speech, image, request, and audio response data are matched to the correct session, reducing context mismatch during processing
Vision and VLM	Provide images through the HTTP vision server on ESP32-CAM and combine images with the user’s question so the VLM can infer, describe scenes, answer questions, find/pick objects, and support nearby navigation	The system understands the surrounding context according to the query and generates appropriate responses for each real-world need
Prototype demonstration	Clearly show BLE, HTTP, AI server status, and the audio response flow back to EFR32xG26 in vision QA, object finding/picking, and nearby navigation scenarios	A clear, repeatable, and evaluable demo chain

3. Proposed solution and usage flow

The system solution is organized as a four-component architecture, where each block performs a separate role but is tightly linked in the same interaction session. EFR32xG26 is responsible for wake-word detection, voice command capture, and audio playback for the user. ESP32-CAM provides scene images over an internal HTTP connection to the Android gateway. The Android gateway is an application developed by the team and serves as the main coordinator: it receives audio from EFR32xG26, retrieves images from ESP32-CAM, attaches session metadata, and sends a unified request to the AI server. On the AI server, the coordination agent uses a local VLM hosted via Ollama to choose or switch between specialized state machines such as scene description, object retrieval assistance, and route guidance; ASR, YOLOE, MediaPipe, and the VLM provide the input data used to generate context-appropriate responses.

A session is executed in a clear order. When EFR32xG26 detects the correct wake word, the device emits a short beep to indicate that recording has started. The voice command is captured

until the audio level stays below the endpoint threshold for a sufficient duration; then the device emits another beep to indicate that recording has stopped and sends the voice data via BLE to the Android gateway app. The Android app then calls `/capture` to obtain an image from ESP32-CAM, attaches `session_id` and timestamps, and sends the unified request via HTTP to the AI server. EFR32xG26 and ESP32-CAM do not communicate directly with the AI server. At the AI server, the agent uses the local Ollama VLM to determine the user's request and routes it to the appropriate state machine. After the result is generated, the audio response is sent back through BLE to EFR32xG26 to be played for the user.

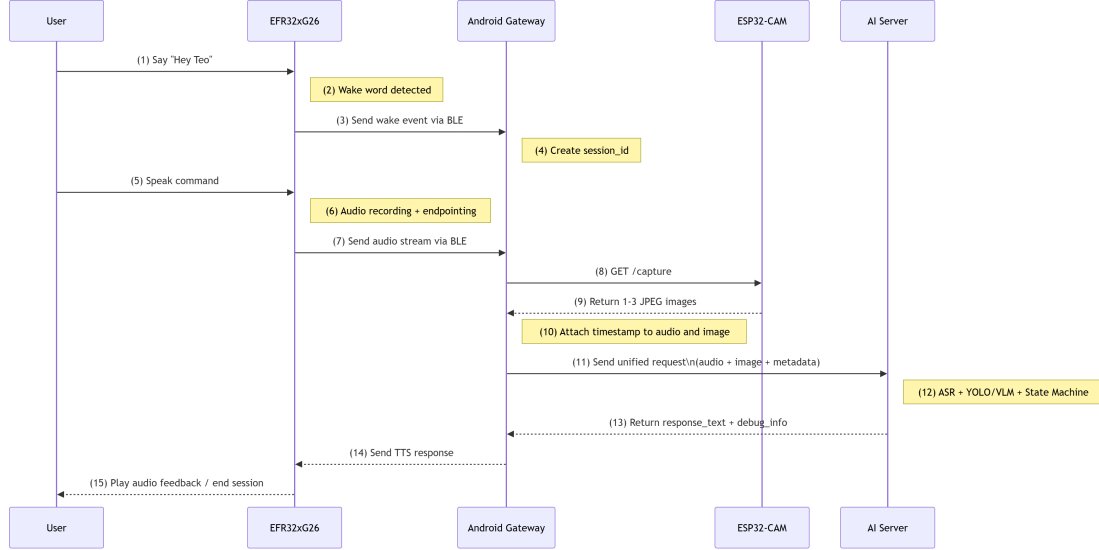


Figure 1: Illustration of the processing flow in the proposed solution and usage flow. The figure shows that both EFR32xG26 and ESP32-CAM send data to the Android gateway; then only the Android gateway sends a unified request to the AI server and returns the audio response to EFR32xG26 for the user.

The first scenario is vision-based question answering about the scene in front. The user says "Hey Tèo, what is in front?" or "Hey Tèo, is there a door ahead?" The edge AI node detects the wake word, opens a recording session, packages the voice command, and sends it to the Android gateway. The Android app calls `/capture` to obtain a JPEG image at the correct time from ESP32-CAM, then the gateway sends the audio command and image to the AI server. ASR converts the audio into text; the VLM combines the question with the image to infer the appropriate answer; the coordinator generates the final response; and the Android gateway sends the audio response back through BLE for EFR32xG26 to play.

The second scenario is finding and assisting with object pickup. The user says "Hey Tèo, find the water bottle" or "Hey Tèo, help me get the medicine box." The voice command from EFR32xG26 and the JPEG image from ESP32-CAM are merged by the Android gateway within the same session and then sent to the AI server. In this scenario, YOLOE is used to detect the target object according to the request, while MediaPipe is used to detect the user's hand and estimate the direction of reaching within the frame. From these two sources of information, the system can provide more specific guidance such as moving the hand left, right, higher, lower, or closer to touch the required object. When necessary, the VLM adds contextual description to clarify where the object is relative to the user's hand. The response is then sent through BLE so EFR32xG26 can speak it to the user.

The third scenario is nearby navigation guidance. The user may ask "Hey Tèo, which direction is safe?" or "Hey Tèo, can I go straight?" In this case, the VLM does not only describe a

single image but also combines the image with the user’s directional question to extract spatial context such as the open space ahead, the direction of the path, the location of prominent obstacles, or corridor edges. The system then generates a short response such as “you can go straight,” “slightly to the left,” or “there is an obstacle ahead, so move to the right,” and sends that response through BLE for EFR32xG26 to speak.

Table 2: Processing flow of one interaction session.

Step	Actor	Action	Generated data	Outcome
1	User	Calls the wake word and speaks the command	Input speech signal	Interaction session is initialized
2	EFR32xG26	Detects wake word, opens audio session, endpointing	Wake event, audio command, device status	Speech data is finalized by session
3	Android gateway	Receives audio command via BLE and creates <code>session_id</code>	Session metadata, timestamp	Session is centrally managed
4	Android gateway	Calls <code>/capture</code> to ESP32-CAM	1–3 JPEG images, image timestamp	Vision data is attached to the session
5	AI server	Processes the unified request	<code>recognized_text</code> , <code>detected_objects</code> , <code>response_text</code>	Multimodal processing result is generated
6	Android gateway + EFR32xG26	Gateway transfers response via BLE, EFR32xG26 plays audio	Response packet, session status	Session ends after the response is completed

4. System architecture

At the architectural level, the prototype is divided into four main processing nodes: the edge speech node on EFR32xG26, the vision node on ESP32-CAM, the Android gateway on the phone, and the AI server for heavy computational tasks. This separation allows each hardware component to perform a clear role while still participating in the same interaction session managed by `session_id`.

Within this report, the term Android gateway is used consistently to refer to the intermediate application running on the Android phone. EFR32xG26 handles wake-word activation, voice command capture, audio transmission via BLE to the gateway, and playback of the final audio response for the user. ESP32-CAM serves as the vision node, providing JPEG images on request through an internal HTTP server to the gateway. The Android gateway coordinates the session, attaches `session_id` and `timestamp`, merges audio and image data, sends the processing request to the AI server via HTTP, and sends the response back to EFR32xG26. Thus, EFR32xG26 and ESP32-CAM do not send data directly to the AI server; only the Android gateway communicates with the AI server. The AI server performs speech recognition, object detection using YOLOE, hand detection using MediaPipe, contextual analysis using a vision-language model, and generates a response aligned with the user’s query.

Table 3: Functional separation among the nodes in the system architecture.

Layer / Node	Hardware	Main function	Input data	Output data	Protocol
Edge speech AI node	EFR32xG26 Dev Kit	Wake-word detection, command capture, endpointing, audio response playback	Microphone background audio, feedback packets from gateway	Wake event, audio command, device status, audio feedback	BLE
Internal HTTP vision node	ESP32-CAM + camera	Provide JPEG images and camera status on request	HTTP request from Android gateway	JPEG frame, camera status, stream debug	HTTP over internal Wi-Fi
Android gateway	Android phone	Session management, audio-image synchronization, AI server calls, response transfer to EFR32xG26	Wake event, audio command, JPEG frame	Unified request, response packet, feedback status	BLE, HTTP, Wi-Fi/Internet
AI server node	PC / laptop running AI server	Process ASR, YOLO/VLM, and response coordination	Audio + image + session metadata from Android gateway	recognized_text , detected_objects , response_text	API with Android gateway over Wi-Fi/Internet

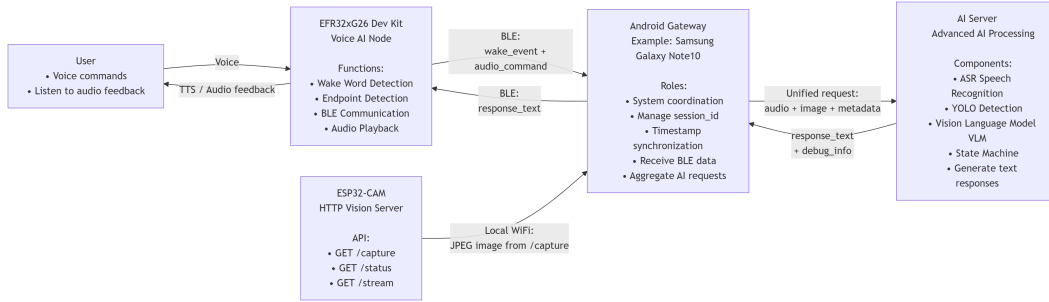


Figure 2: Overall system architecture diagram. EFR32xG26 sends speech data via BLE to the Android gateway, ESP32-CAM provides images through the HTTP Vision Server to the Android gateway, and the Android gateway is the only component that sends a unified request to the AI server for ASR, YOLO/VLM processing, and response coordination.

4.1 Speech processing with edge AI on EFR32xG26

EFR32xG26 Dev Kit is the edge AI node responsible for the entire speech path on the wearable device. The purpose of this node is not full language processing, but rather detecting the exact moment the user calls the system, capturing the command after the wake word, identifying the end of speech, and transmitting the packaged audio data to the Android gateway. This structure prevents the device from streaming audio continuously, reduces communication load, and still keeps the interaction natural for the user.

The wake-word model is trained specifically for the activation phrase “Hey Tèo” as a binary keyword spotting task. The dataset is organized similarly to Google Speech Commands v2 and Silicon Labs xG26 audio classification examples: each class is stored in a separate folder, where **hey_teo** is the target wake-word class, **_unknown_** contains non-wake-word samples and hard negatives, and **_silence_** contains silence/background noise.

```

dataset/
├── hey_teo/
│   ├── sample001.wav
│   ├── sample002.wav
│   └── ...
├── _unknown_/
│   ├── neg001.wav
│   ├── neg002.wav
│   └── ...
└── _silence_/
    ├── noise001.wav
    ├── noise002.wav
    └── ...

```

Here, `hey_teo` includes segments containing “Hey Tèo” or close variants such as “Hey Tèo ơi”; `_unknown_` includes ordinary conversations, speech from TV/YouTube, and phonetically similar words such as “hey”, “tèo”, “theo”, and “hello”; while `_silence_` includes fan noise, classroom noise, traffic noise, wind, white noise, or silent segments. Adding hard-negative samples in `_unknown_` is especially important because Vietnamese has tonal accents and many similar sounds, which can easily cause false activation if the model is trained only with clean data.

The recorded audio is normalized to mono PCM 16-bit WAV, sampled at 16 kHz, and about 1–2 seconds long per sample to fit the keyword spotting pipeline on EFR32xG26. The data sources include phones, laptop microphones, and microphones used with EFR32xG26 to increase similarity with real deployment conditions. During data preparation, the team applies augmentation such as adding background noise, time shifting, and slightly changing pitch or speaking speed, while balancing the data among `hey_teo`, `_unknown_`, and `_silence_` so that the model does not become biased toward one class.

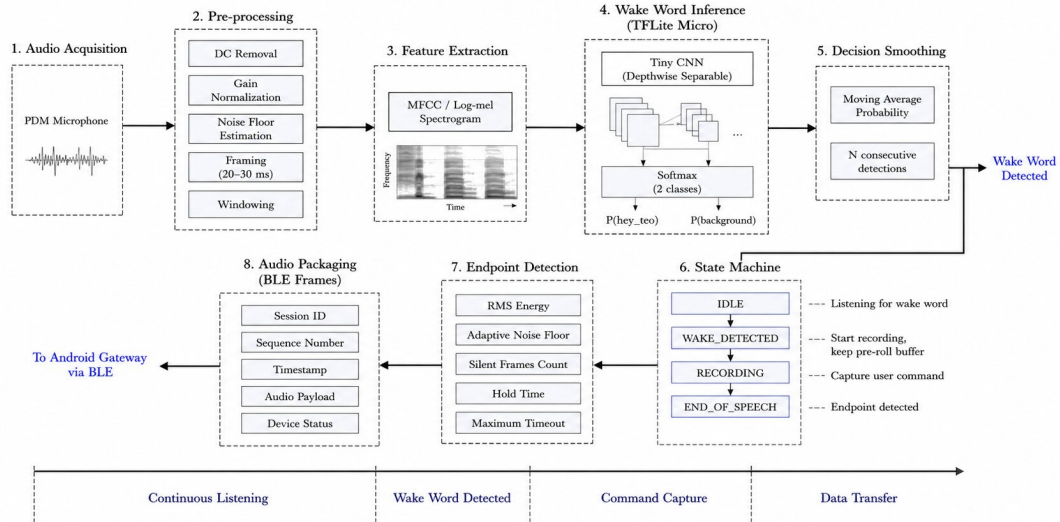


Figure 3: Training and deployment pipeline for the “Hey Tèo” wake word on EFR32xG26. The figure illustrates the workflow from dataset organization following the Speech Commands structure, MFCC/log-mel feature extraction, keyword spotting model training, INT8 quantization, to real-time inference on firmware.

Before entering the TinyML model, the audio signal is segmented into short frames of about 20–30 ms and converted into MFCC/log-mel spectrogram features according to the training pipeline configuration. These features are the input to a compact keyword spotting model

using Depthwise Separable CNN, a Dense layer, and a two-class Softmax of **background** and **hey_teo**. Training is carried out with TensorFlow/Keras, TensorFlow Lite, and the Silicon Labs Machine Learning Toolkit; afterward the model is evaluated in TFLite form to check accuracy, false positive rate, and false negative rate before deployment to the device.

After reaching the desired quality, the model is quantized to INT8 to reduce Flash, RAM, and latency, then the `.tflite` file is converted into a C/C++ array embedded into the EFR32xG26 firmware. During runtime, the firmware samples the microphone, extracts MFCC/log-mel features, and performs inference with TFLite Micro to compute the wake-word probability; the result is stabilized using a moving average or by requiring multiple consecutive threshold crossings before switching from **IDLE** to **WAKE_DETECTED**. After the start-of-recording signal, the device keeps a pre-roll buffer, switches to **RECORDING**, and uses endpointing based on energy, noise floor, silence-frame count, hold time, and timeout to identify **END_OF_SPEECH**. It then splits the audio command into BLE frames with `sequence_number` and sends them together with device status to the Android gateway for synchronization with the ESP32-CAM image before building the AI request.

4.2 Communication protocol and HTTP Vision Server

The system uses a multi-layer communication architecture: BLE transfers the audio command from EFR32xG26 with low energy usage, HTTP over the internal network fetches JPEG images from ESP32-CAM, and the Android gateway coordinates the session, merges multimodal data, and is the only component that sends requests to the AI server. This separation allows the edge nodes to operate independently while still remaining synchronized by session, and it centralizes timestamp, session state, retry policy, and logging on the gateway.

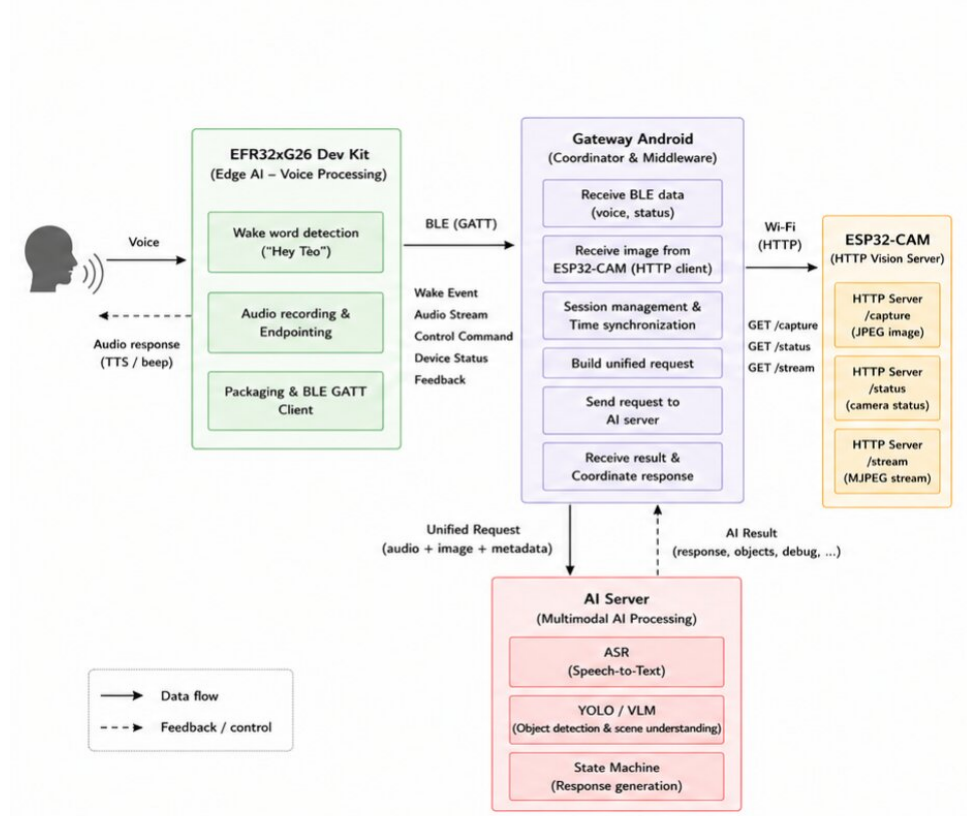


Figure 4: Multi-layer communication architecture of the system. EFR32xG26 and ESP32-CAM only exchange data with the Android gateway; the gateway is the sole entry point that creates the unified request and communicates with the AI server.

4.2.1 BLE GATT between EFR32xG26 and the Android gateway

The BLE channel between EFR32xG26 and the Android gateway is built using a BLE GATT Service model to optimize low-power communication and event-based data transfer. Instead of streaming the microphone continuously, the edge AI node only transmits data when the wake word is confirmed and the audio command has been recorded. This event-driven approach reduces power consumption, lowers BLE bandwidth load, and improves user privacy.

The characteristics are separated by function to simplify testing, debugging, and session lifecycle management. Here, Wake Event is used to start an interaction session; Audio Stream transfers the audio command in BLE frames; Control manages session state; Device Status monitors the state of the edge AI node; and Feedback Command allows the gateway to send the processing result back to the wearable device.

For Audio Stream, the audio command is fragmented into multiple small BLE packets to fit BLE MTU limits. Each packet contains `sequence_number`, timestamp, and payload length to support packet loss detection, frame reordering, and debug logging. The Android gateway performs packet reassembly before creating the final audio blob for the AI server.

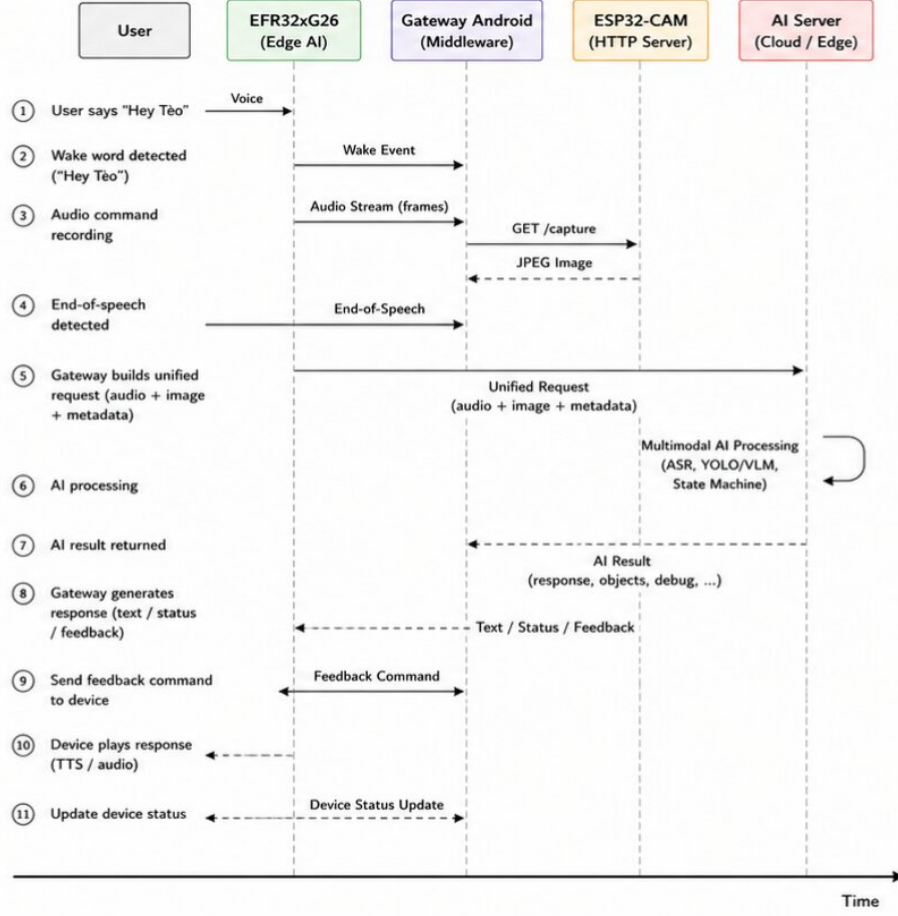


Figure 2. Interaction sequence from wake word to system response.

Figure 5: Session lifecycle at the Android gateway. The gateway receives the audio command from EFR32xG26, fetches a JPEG image from ESP32-CAM, attaches time metadata, sends the unified request to the AI server, and returns the response to the wearable device via BLE.

4.2.2 HTTP Vision Server on ESP32-CAM

ESP32-CAM acts as an independent vision node within the local network and provides image data using a request-response mechanism. The device is configured in Wi-Fi station mode and connects to the Android gateway hotspot or a local router to reduce image transmission latency. After the camera driver is initialized and the network connection is established, ESP32-CAM creates an HTTP Vision Server with three main endpoints: `/capture`, `/status`, and `/stream`. Among them, `/capture` is the most important endpoint in the AI pipeline because it provides a single JPEG frame at the exact time the gateway requests it. This allows the system to synchronize images with the audio command by session instead of maintaining continuous video streaming.

The `/status` endpoint allows the Android gateway to check the vision node status such as camera readiness, current FPS, heap usage, or Wi-Fi RSSI before sending the AI request. The `/stream` endpoint provides an MJPEG stream for live observation and debugging during development, but it is not part of the main AI pipeline because it consumes more bandwidth.

4.2.3 Unified request from the Android gateway to the AI server

The Android gateway is an application developed by the team and serves as the central coordination layer of the entire system. After receiving the audio command from EFR32xG26 and the JPEG image from ESP32-CAM, the app creates a unified request for each session before sending it to the AI server.

The request is organized as a multimodal request, including session metadata, audio command, and image payload in the same temporal context. Important fields include `session_id`, `wake_timestamp`, `audio_start_timestamp`, `audio_end_timestamp`, `image_timestamp`, audio blob, JPEG image, and `device_status`. With this structure, the AI server can accurately synchronize the user's speech with the scene state at the same moment.

On the AI server, the agent uses a local VLM hosted via Ollama to read the question, image, and session metadata, then chooses the appropriate state machine for tasks such as object pickup assistance, route guidance, or scene description. After the corresponding state machine finishes processing, the AI server returns recognized text, vision results, response content, priority level, and debug information; the Android gateway uses these results to send feedback to EFR32xG26 via BLE and end the session.

4.2.4 Time synchronization and session management

Each interaction session is identified by a `session_id` created by the Android gateway immediately after the wake event is confirmed. The gateway acts as the central time source of the system to avoid timestamp drift among the edge AI nodes.

Important timestamps such as wake timestamp, audio start, audio end, and image capture time are all recorded at the Android gateway to create a unified time reference. This is especially important for multimodal queries that need to relate spoken commands to the corresponding image frame.

To improve stability, the gateway sends the AI request only when both the audio command and the valid JPEG frame are ready. If the `/capture` endpoint fails or times out, the gateway retries within a limited number of attempts. In case BLE audio misses frames or packets are lost, the session is marked as `incomplete` in the debug log for later checking and error analysis after the demo or testing phase.

4.3 Hardware components

The hardware architecture is designed with clear separation among speech, vision, session coordination, and AI processing functions in order to reduce the load on the wearable device and improve the scalability of the prototype. EFR32xG26 handles edge AI for the speech path, ESP32-CAM handles vision data, the Android gateway coordinates multimodal sessions, and the AI server performs deep inference tasks.

Table 4: Hardware components and their roles in the prototype.

No.	Component	Role in the system	Main communication / data	Status
1	EFR32xG26 Dev Kit	Edge AI node for wake word, endpointing, BLE audio, and audio playback	Wake event, audio command, BLE GATT, I2S audio	Available
2	Stereo MEMS microphone (ICS-43434 on xG26)	Captures speech signals for the TinyML pipeline	16 kHz PCM audio directly into firmware inference	Integrated on board
3	MAX98357A I2S Amplifier	Decodes and amplifies the response audio signal	I2S audio from EFR32xG26 to speaker	Available
4	ESP32-CAM	HTTP Vision Server and vision node of the system	JPEG image, HTTP endpoints /capture, /status, /stream	Available
5	OV2640 camera (integrated on ESP32-CAM)	Captures the user-facing environment image	Internal camera frames for ESP32-CAM	Integrated on board
6	Android gateway (prototype: Samsung Galaxy Note10)	Android application that coordinates sessions and synchronizes multimodal data	BLE with xG26, HTTP with ESP32-CAM, Wi-Fi/Internet with AI server	Available
7	AI Server	Runs ASR, YOLO/VLM, Ollama agent, and task-specific state machines	Unified request, local VLM via Ollama, GPU/CPU server	Available
8	ICM-40627 6-axis IMU (integrated on xG26)	Collects inertial data for extended contextual awareness	Accelerometer and gyroscope	Integrated on board
9	Power supply / battery	Supplies power to wearable nodes	Internal DC power	In progress
10	Glasses frame / mechanical structure	Integrates and fixes hardware modules on the wearable prototype	Mechanical mounting space	In progress

4.4 Software and AI flow

The software flow is organized in an event-driven manner combined with a state machine so that the edge AI, Android gateway, and AI server are synchronized across each interaction session. Each component handles only a specific group of functions, which makes the system easier to test, extend, and debug during development.

In the **IDLE** state, EFR32xG26 continuously runs the TinyML model to detect the wake word “Hey Tèo.” When the wake word is confirmed, the state machine transitions to **WAKE_DETECTED** and sends a wake event to the Android gateway via BLE GATT.

The system then moves to **RECORDING**. EFR32xG26 starts recording the audio command, keeps a pre-roll buffer to avoid losing the beginning of the command, and splits the audio into multiple

small BLE frames with `sequence_number`. The Android gateway performs packet reassembly to build the complete audio for each session.

At the same time, the Android gateway sends an HTTP request to ESP32-CAM via the `/capture` endpoint to retrieve the JPEG image corresponding to the moment when the user issued the command. ESP32-CAM acts only as an internal HTTP Vision Server and does not communicate directly with the AI server.

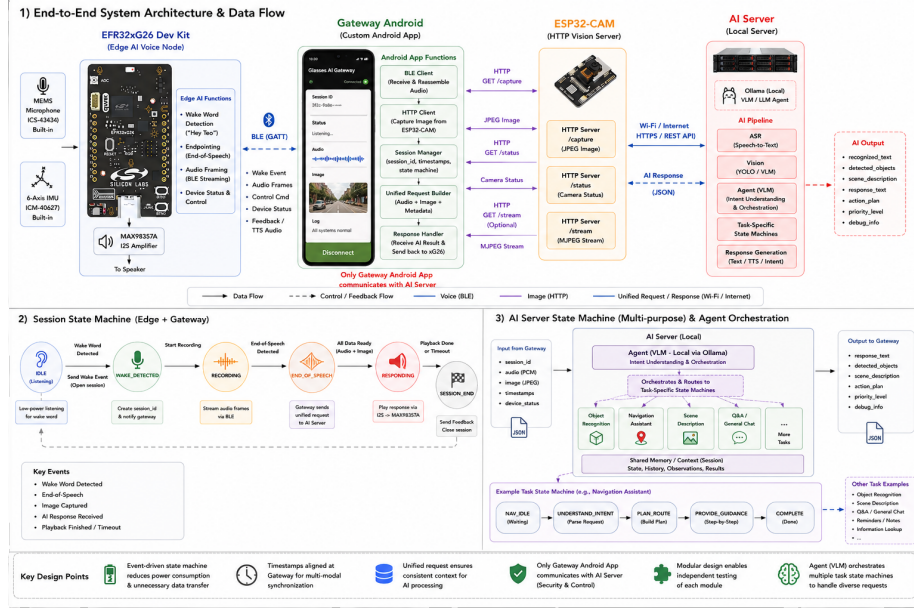


Figure 6: Session lifecycle at the Android gateway. The gateway receives the audio command from EFR32xG26, fetches a JPEG image from ESP32-CAM, attaches time metadata, sends the unified request to the AI server, and returns the response to the wearable device via BLE.

The Android gateway acts as the central middleware of the entire system. The Android app manages `session_id`, synchronizes timestamps between audio and image, and then creates a unified request containing audio, image, and metadata to send to the AI server.

At the AI server, the audio command is processed by ASR to produce `recognized_text`, while the JPEG image is sent through YOLO or a Vision Language Model to produce `detected_objects` and `scene_description`. A VLM hosted locally with Ollama acts as the coordinator between task-specific state machines such as object finding assistance, scene description, or movement guidance.

After processing is completed, the AI server returns `response_text` to the Android gateway. The response is then sent via BLE to EFR32xG26 for audio playback through MAX98357A and an external speaker. When the session ends, the state machine returns to IDLE to continue listening for the next wake word.

Overall, the lifecycle of one interaction session can be described as follows:

IDLE → WAKE_DETECTED → RECORDING → PROCESSING → RESPONDING → IDLE

5. Resources, assignment, and schedule

The task allocation follows four main implementation directions: system coordination and design, EFR32xG26 embedded development, Android gateway app development, and AI server

development. Tasks are assigned directly to each member to avoid overlap and to make the full-chain integration easier to track. Since the modules can be developed in parallel, the schedule is organized into concurrent work items with a full completion milestone in September.

Table 5: Task assignment by member.

Member	Main responsibility	Tasks and deliverables
Ha Duc Khoi	Team leader, hardware framework and AI design	Finalize the overall architecture, coordinate integration, design the glasses frame / hardware mounting structure, build the AI Processing API, Ollama agent, and task-specific state machines. Deliverables are the system architecture, prototype hardware frame, and AI server pipeline.
Nguyen Huong Giang	EFR32xG26 embedded: wake word and endpointing	Implement the TinyML pipeline on xG26, microphone sampling, wake word detection for “Hey Tèo”, end-of-speech detection, and stable audio session opening/closing. Deliverable is a speech node that works correctly within the session lifecycle.
Nguyen Minh Cat	EFR32xG26 embedded: BLE and audio response	Develop BLE GATT, package audio frames with <code>sequence_number</code> , receive feedback commands from the gateway, and play audio through I2S/MAX98357A. Deliverable is the two-way audio and response flow between xG26 and the gateway.
Kieu Nhat Linh	Android gateway app	Code the Android app to manage <code>session_id</code> , timestamp, BLE with xG26, HTTP with ESP32-CAM, send unified requests to the AI server, and return responses to the wearable device. Deliverable is a fully functional Android gateway app.

Table 6: Implementation schedule from May to September.

Item	Implementation period	Main content	Completion milestone
Finalize overall architecture	May	Define the xG26-Android-ESP32-CAM-AI server flow, standardize <code>session_id</code> , API schema, endpoints, and a preliminary state machine.	System design document completed.
Hardware frame and module placement	May-July	Design, test, and refine the glasses frame; arrange power supply, camera, microphone, and speaker so the prototype can be worn.	Prototype frame fixed with all modules.
EFR32xG26 embedded development	May-August	Standardize wake-word data, build the TinyML pipeline; integrate wake word “Hey Teo”, endpointing, audio buffer, BLE frames with <code>sequence_number</code> , and I2S/MAX98357A playback.	Firmware ready for demo integration.
ESP32-CAM and HTTP Vision Server	May-June	Implement <code>/capture</code> , <code>/status</code> , <code>/stream</code> ; stabilize JPEG capture, configure internal Wi-Fi, and test basic latency.	Camera server completed early for gateway integration from June.
Android gateway app	May-August	Build status UI, manage <code>session_id</code> , integrate BLE with xG26, HTTP with ESP32-CAM, create unified request, log timestamps, and implement retry policy.	App can coordinate one complete session.
AI server and Ollama agent	May-August	Complete ASR/YOLO/VLM pipeline, integrate local VLM via Ollama, build the coordination agent, fallback logic, and debug logging for main tasks.	API returns stable responses to the gateway.
Full-chain integration and testing	July-September	Connect all nodes in one session, repeatedly run scene description, object finding, and route guidance scenarios; measure latency, stability, and fix interoperability issues.	Demo completed and ready for defense.

6. Current results and progress of the prototype implementation

This section reflects the actual current status of the prototype rather than presenting it as a system that has already fully integrated all four nodes. At the time of writing, the team has completed a rough version for the object-pickup assistance scenario. In this version, ESP32-CAM captures an image and sends it directly to the AI server; the server processes the image, identifies the target object in the frame, and generates experimental guidance. This workflow allowed the team to verify the vision and AI parts first before adding the Android gateway and EFR32xG26 node.

In addition to the image path, the experimental voice flow is also organized as direct transmission to the server. Audio is captured from the INMP441 microphone attached to ESP, and then ESP sends the audio data directly to the AI server for STT (Speech-to-Text). The server uses the recognized command content together with the image from ESP32-CAM to process the object-pickup scenario, then generates an answer and performs TTS (Text-to-Speech). The audio response is sent back to be played through the speaker module mentioned in the hardware section, specifically through I2S/MAX98357A.

At present, EFR32xG26 and the Android gateway application are not yet integrated into the

real execution flow. Therefore, the current demo does not include the wake word “Hey Tèo”, endpointing, BLE GATT, `session_id` management on the Android app, or audio playback through EFR32xG26. These components remain part of the final architecture and will be integrated after the ESP–AI server flow becomes stable.

ESP32-CAM/ESP + INMP441 → AI SERVER (STT + Vision + TTS) → I2S/MAX98357A
speaker

Table 7: Current progress of the prototype.

Item	Current status	What is already available	Next step
Physical glasses prototype	Rough version available	A glasses-like prototype has been assembled to mount the camera and check the real viewing direction.	Fix the modules, optimize wiring, power supply, and wearing position.
ESP32-CAM	Rough implementation running	Captures images and sends data directly to the AI server in the test scenario.	Stabilize Wi-Fi, power supply, camera angle, and response time.
AI server for object-pickup scenario	Rough implementation running	Receives images from ESP32-CAM, processes the object-finding/pickup scenario, and generates test guidance for the user.	Refine object detection, position description, and guidance responses.
ESP32-CAM–AI server flow	Rough end-to-end demo available	The current flow goes directly from ESP32-CAM to the server, proving that the vision–AI part works first.	Switch to the standard architecture through the Android gateway app to synchronize sessions.
EFR32xG26	Not yet integrated into the current demo	No wake word, endpointing, BLE, or audio response playback in the real execution flow yet.	Integrate the speech node, BLE GATT, and audio playback after the vision–server demo step.
Android gateway app	Not yet integrated into the current demo	No <code>session_id</code> management, BLE with xG26, HTTP with ESP32-CAM, or unified request to the AI server in the current demo.	Build the gateway app to replace the direct ESP32-CAM-to-server flow.

Current source code and images: The current demo source code is located at the [GitHub repository of the current demo](#). The figures below record the running demo and the rough physical glasses prototype.

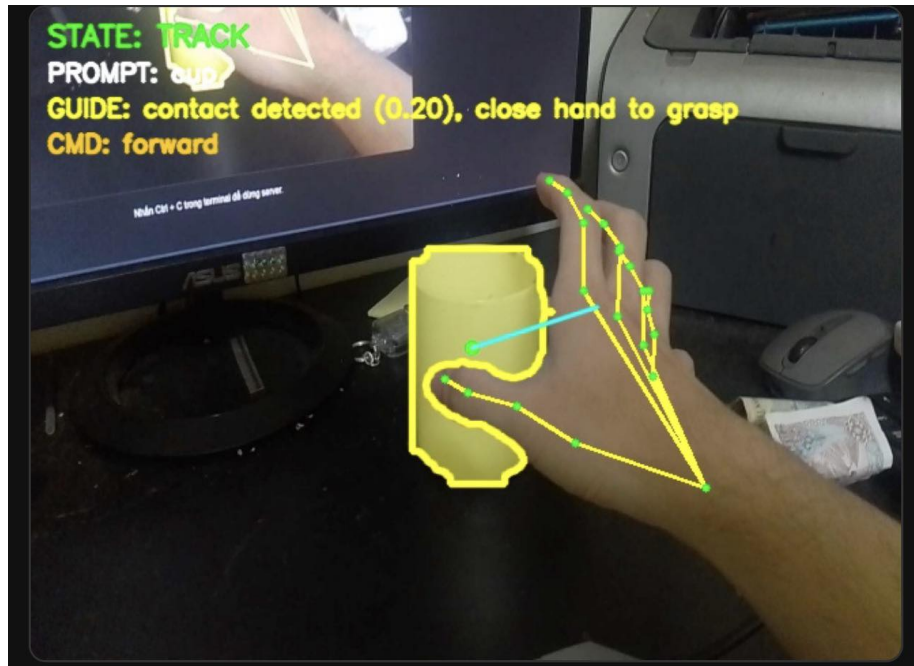


Figure 7: Demo of the ESP32-CAM direct-to-AI-server flow for the object-pickup assistance scenario.



Figure 8: Photo of the rough glasses prototype used to check camera placement, viewing direction, and wearability.

7. Conclusion

This project addresses a clear social problem: helping blind users perceive the nearby environment using a wearable system that can receive voice commands and respond with audio. A multi-node architecture has been defined with specific roles for EFR32xG26, ESP32-CAM, Android gateway, and AI server. Each component processes a separate data layer, while all of them participate in a unified session chain from wake word to audio response.

With this organization, the prototype aligns directly with the criteria of Wireless Technology, Edge AI, IoT/Embedded, and AI. The report has defined the technical flow, communication protocol, current progress, and a rough demo for the object-pickup assistance scenario. This provides a suitable foundation for the team to continue integrating EFR32xG26, the Android gateway app, perform full-chain testing, and prepare for the final competition presentation.